```
      }
    }
  }
}
```

Because the `word_count` field is an inner attribute of the `title` field, we can use a `term` query (a type of query run on structured data such as numbers, dates, Booleans, etc.).

Listing 4.13 Searching for books with a four-word title

```
GET tech_books/_search
{
  "query": {
    "term": {
     "title.word_count": {
        "value": 4
      }
    }
  }
}
```

We are using a term query.

The name of the field

We use `<outer_field>.<inner_field>` as the `word_count` field's name, so `title.word_count` is the accessor of the field.

In addition to `token_count`, the `text` type has other descendants, such as `search_as_you_type` and `completion`. For space reasons, we won't discuss them in this book. Let's continue learning about common data types, with `keyword` being next in line.

4.5.2 The keyword data types

The `keyword` family of data types comprises `keyword`, `constant_keyword`, and `wildcard`. Let's look at these types.

THE KEYWORD TYPE

Structured data, such as PIN codes, bank accounts, and phone numbers, doesn't need to be searched as partial matches or produce relevant results. The results tend to provide binary output: a result is returned if there is a match, or no result is returned. This type of query doesn't care how well the document is matched, so we don't expect relevance scores associated with the results. Such structured data is represented as a `keyword` data type in Elasticsearch.

The `keyword` data type leaves fields untouched. The field is untokenized and not analyzed. The advantage of `keyword` fields is that they can be used in data aggregations, range queries, and filtering and sorting operations on data. To set a `keyword` type, use this format:

```
"field_name":{ "type": "keyword" }
```

For example, the following code creates an `email` property with the `keyword` data type.